Outage-Aware Reporting and Loss Recovery for Energy-Efficient River Water-Quality Telemetry over LPWANs

Kofi Sarpong Adu-Manu and Stephen Kwaku Azongo

Abstract

Battery-powered river water-quality nodes lose data, yet retransmission, buffering, and reconstruction are analysed for short, independent losses. This study measures the loss in four real systems: two water-quality deployments and two LoRaWAN networks. Losses fall into short radio losses and rarer long delivery gaps, which hold most of the missing data in the water-quality deployments; a geometric gap-length model fits worst on every record. Replayed through a measurement-parameterised LoRaWAN and NB-IoT energy model, retransmission recovers under a fifth of the missing readings even under optimistic retry timing, at 2.3×–7.3× the energy of unconfirmed reporting on LoRaWAN, whereas buffering with back-off recovers every generated reading at an energy close to that of recovering nothing. Neither simple nor deep reconstruction keeps most day-long gaps within tolerance. Bounded event-driven batching combines hourly batches, immediate alarms, a bounded-error trigger, and buffering with back-off. Tested with frozen rules on held-out data from 33 river sites, it matches per-reading reporting on alarm latency and on the real-time picture, while per-reading retransmission uses 1.34×–2.55× as much energy. Recovery of many nodes after a shared outage requires randomised probes, and large SF12 cells are limited by the gateway's acknowledgements.

Index Terms

Delivery gaps, network outages, store-and-forward, ARQ, LoRaWAN, NB-IoT, energy efficiency, water-quality monitoring, data reconstruction, event detection.

K. S. Adu-Manu and S. K. Azongo are with the Department of Computer Science, University of Ghana, Legon, Ghana. Corresponding author: K. S. Adu-Manu.

I. Introduction

River and reservoir monitoring using in situ sensor nodes must work where field visits are expensive and power is scarce [1], [2]. A typical node samples temperature (T), specific conductance (EC), dissolved oxygen (DO), and pH every 5–30 min and sends the readings over a cellular or low-power wide-area network (LPWAN). Such nodes lose data. The InWaterSense deployment on the Sitnica river in Kosovo missed 46.0% of its expected readings [3]; the CSIRO water-quality network in Australia averaged approximately 64% delivery because the gateway's connection to the server "was down occasionally" [4]; and deployment reports describe gateways that lose power every night [5], sinks that stop reconnecting until rebooted [6], [7], and outages of days to weeks [8], [9].

A node can retransmit a lost reading until it is acknowledged (automatic repeat request, ARQ), buffer it and backfill it later, or leave the server to reconstruct it from the site's history or its neighbours. The correct response depends on how data are lost: retransmission helps against a short radio fade but wastes energy against a gateway that is down for a day, and reconstruction can fill ten minutes but not a week. Yet, the analyses behind these remedies assume short losses: estimation over lossy links uses independent or short-memory (Markov fading) packet errors [10]–[12], and imputation methods are evaluated on random masks or simulated failures of a few hours [13]–[16]. How long real delivery gaps last, how much of the missing data they hold, and what this implies for the energy and timeliness of each remedy have not been measured for environmental telemetry.

The contribution of this study is not a new retransmission or prediction primitive; ARQ, buffering, back-off, batching, and error-bounded (dual-prediction) reporting are established. It is an evidence-to-design chain for outage-aware, energy-efficient telemetry. Real delivery records measure how long losses last and where the missing data are. The records are replayed through a measurement-parameterised LPWAN energy model, with explicit bounds on what they cannot show (why a reading is missing, and when within a slot the network returned), to decide when each remedy pays. The resulting reporting design is fixed on one year of river data and tested on a held-out year. Four research questions structure the paper:

RQ1 In the records studied, how long are the delivery gaps, how much of the missing data do long gaps hold, and is a geometric (memoryless) gap-length model adequate?

RQ2 Under these gaps, what do retransmission, buffering with back-off, and batching recover, at what energy and delay, on LoRaWAN and NB-IoT?

RQ3 Can reconstruction from a site's own history or its neighbours replace delivery during a gap?

RQ4 Can one reporting scheme raise alarms as fast as per-reading reporting and keep the server's real-time picture within a stated tolerance as often, at substantially less energy than per-reading retransmission, on data not used for its design?

The contributions are as follows:

- Empirical loss structure (Section IV): In four records, two water-quality deployments and two LoRaWAN networks, losses fall into short radio losses and rare long delivery gaps, which hold most of the missing readings in the water-quality records. A geometric gap-length model is strongly unsupported relative to the alternatives and fails worst in the long-gap tail. Where the records allow it, the cause of long gaps is traced.
- Consequence analysis with explicit bounds (Sections V and VI): ARQ recovers under a fifth of the missing readings even under optimistic retry timing, at 2.3×–7.3× the energy of unconfirmed reporting on LoRaWAN, whereas buffering with back-off recovers every generated reading at an energy close to that of recovering nothing. The results are bracketed for the unknown cause of gaps, retry timing, storage, the duty cycle, model uncertainty, and many nodes recovering simultaneously. The evaluated reconstruction estimators, including two deep imputers, keep few day-long gaps within tolerance.
- An outage-aware reporting design, confirmed on held-out data (Section VII): Bounded event-driven batching combines hourly batches, immediate alarm uploads, a bounded-error trigger that mirrors the server's estimate, and buffering with back-off. It was developed at 30 USGS river sites in 2023 and tested, with its rules frozen, at 33 sites in 2024.

Scope: The two water-quality records cannot fully separate a network outage from a node that stopped producing readings; therefore, the paper speaks of delivery gaps and treats buffering's recovery as an upper bound. Energy comes from a model parameterised by published measure-

ments, not from hardware built for this study, and the claims are ratios between strategies on the same radio. A supplement provides the details behind each section.

Section II reviews related work. Section III describes the data and models. Sections IV–VII answer RQ1–RQ4, and Section VIII discusses guidance and limitations.

II. Related Work

How sensor networks lose data: Packet delivery in dense sensor networks varies widely between links and over time [17]. Kong et al. [13] analysed three real datasets with losses of 23–64% and identified five loss patterns, from random loss to "successive loss in a row" when a node fails, without measuring durations. Deployment reports describe long gaps anecdotally: a gateway losing its server connection [4], nightly gateway power loss [5], sinks and modems that stop until reset [6], [7], and LoRaWAN outages of days to weeks [8]. Public LoRaWAN datasets now allow loss to be measured from frame counters: LoED [18] and a 16-month deployment at the University of Virginia (UVA) [19].

Estimation and retransmission: Kalman filtering with intermittent observations was analysed under independent arrivals [10]; remote estimation asks whether to retransmit a failed packet or send a fresh one under independent or Markov fading channels [11], [12], and the age of information formalises timeliness [20], [21]. Application-layer coding recovers LoRaWAN frames lost to the radio [22]. None of these solutions addresses periods of days with no communication.

Reconstruction: Compressive and low-rank gathering [23]–[25], and deep spatio-temporal imputation with graph networks [14], [15], [26], [27] or self-attention [28], [29], exploit spatial and temporal correlation. Their evaluations use random losses [16] or simulated blocks of 1–4 h [14], [15], without a per-site diel baseline, although many such signals have strong day–night cycles. River networks have a specific spatial structure along the flow [30].

Store-and-forward and energy: Node-side buffering is an established practice [4], [5], [8], [31], [32], as is gateway-side storage during backhaul outages [9]; reports also note that draining a backlog can starve live traffic [6] and that storage fails [32]. LPWAN energy rests on measured LoRaWAN [33] and NB-IoT [34], [35] profiles and models [36], [37]. Dual prediction transmits only when the server's prediction would leave a tolerance [38], [39], and it was recently combined

Table I
Closest related work. Gap durations: measured from data, reported without measurement, assumed in a model, or simulated. All four: ARQ, buffering, batching, and reconstruction. –: not addressed.

Work	Gap durations	Remedies compared	Energy	Held-out test
Kong [13]	reported	reconstruction	–	–
Deployments [4], [6], [8], [32]	reported	buffering	–	–
Huang [11], [12]	assumed	retransmission	–	–
GRIN, SPIN [14], [15]	simulated	reconstruction	–	–
Arnaiz [40]	–	dual prediction	modelled	–
This study	measured	all four	ratios	yes

with adaptive sampling under an energy budget [40]; it is used to suppress reports over reliable or independently lossy links.

What is missing is a quantitative comparison of these remedies in terms of energy, completeness, and timeliness under measured gap processes (Table I). This study also reuses dual prediction for a different purpose: to make an hourly batch leave early when the server's estimate would leave tolerance.

III. Data and Models

The following terms are used throughout:

- Reading: one sampling cycle of a node, i.e., the four parameters taken at one instant, which travel together. Time is divided into slots at the reporting cadence, one reading per slot.
- Delivery gap: a run of slots for which no reading reaches the server, regardless of the cause.
- System outage: a delivery gap verified to affect every node of a network (its gateway, backhaul, or server).
- Radio loss: a frame lost while the rest of the path works.

A. Delivery records and the cause of gaps

The study uses four records of what a network delivered (Table II; details in Supplementary Section S1):

- InWaterSense (Kosovo): node 1 on the Sitnica river [3], [41], reporting approximately every 611 s from May 2015 to January 2016.

Table II
Delivery records. Loss: share of expected readings (InWaterSense, Weija) or frames (UVA, LoED) that did not arrive. Gaps: runs of missing slots or frames. Lost in $\geq$1 d: $S(1\ \mathrm{d})$ in (1). Weija: conductivity stream (71.0%–100.0% over all nine streams). LoED: well-heard devices (radio loss).

Record	Type	Slot	Loss	Gaps	Lost in $\geq$1 d
InWaterSense	water quality	611 s	46.0%	1,619	50.7%
Weija	water quality	32.5 min	15.4%	42	82.9%
UVA	LoRaWAN, 3 GW	5 min	4.5%	36,519	23.6%
LoED	LoRaWAN, 9 GW	varies	9.6%	56,276	–

- Weija (Ghana): nine parameter streams of one node, October 2018 to March 2019 [2]. Records marked as imputed do not count, and gap statistics use the conductivity stream (32.5-min cadence).
- UVA (USA): ten LoRaWAN sensors reporting every 5 min through three gateways, February 2024 to June 2025 [19], [42]. Missing frames are found from frame counters and classed as system losses if no sensor reached the database in that 10-min bin, as gateway losses if all of the sensor's gateways were silent, and as radio losses otherwise.
- LoED (UK): uplinks heard by nine gateways [18], [43], of which only well-heard devices (at least 200 frames, radio loss below 20%) are used.

Neither water-quality record has a node status log; therefore, a gap in which the network was down cannot be distinguished directly from one in which the node produced nothing. Two partial checks are possible. On Weija, the node's other streams share its logger and uplink: of the 916 missing conductivity slots, 84% were silences of the whole node, and in gaps of 6 h or more, only 6% of the slots had another stream delivering. The long gaps are node-wide silences (node, logger, or uplink). InWaterSense's two nodes reached the server through one gateway and one GPRS link [3]. While node 2 was operating, it was silent in 79% of node 1's missing slots, and after correcting for its own silence, gaps of 6 h or more were approximately 74% shared-cause (gateway, backhaul, server, or shared power) and 26% specific to node 1. Most long gaps are therefore failures of the path to the server, but some may be nodes that produced nothing; Section V treats buffering's recovery as an upper bound.

B. Gap-length models

A record is reduced to alternating up-runs and gaps of $k_1, \ldots, k_n$ slots; let $h(k)$ be the duration of a gap of k slots. Because loss handling depends on where the missing data are, the main statistic is the share of missing slots in gaps of at least duration L,

$$S(L) = \frac{\sum_{i:\, h(k_i) \geq L} k_i}{\sum_{i=1}^{n} k_i}, \tag{1}$$

with bootstrap 95% confidence intervals [44]. The gap lengths are fitted by maximum likelihood with four models. The first is a geometric model,

$$P_{\mathrm{geo}}(k) = (1 - q)\, q^{k-1}, \tag{2}$$

the gap-length distribution of any loss process with a single loss state, including independent losses and the Gilbert–Elliott channel. The second is a two-geometric mixture,

$$P_{\mathrm{mix}}(k) = w\,(1 - q_1)\, q_1^{k-1} + (1 - w)\,(1 - q_2)\, q_2^{k-1}, \tag{3}$$

a Markov chain with one up state and two loss states (short loss, long gap). The third and fourth are a discrete lognormal and a discrete power law. Models are ranked by Akaike's information criterion and tested absolutely by a bootstrap Kolmogorov–Smirnov test (Supplementary Section S2).

C. Energy model and loss-handling strategies

The node is a Waspmote-class host [45] with NB-IoT (the N211 module with release assistance [34]), LoRaWAN SF7, or LoRaWAN SF12 (EU868). Each transmission attempt is charged the measured energy of its radio and outcome, host time included (Supplementary Section S3). NB-IoT energies come from UDP datagrams to an echo server [34]; therefore, a confirmed report is acknowledged by the application, end to end. A cell search without a network costs the receive power [35] times a search time swept over 2.2–60 s. LoRaWAN energies come from state-by-state current profiles [33]: at SF7, the acknowledgement arrives in the first receive window; therefore, a confirmed success is cheaper than an unconfirmed report; at SF12, it is more

expensive. The acknowledgement comes from the network server. LoRaWAN 1.0.2 recommends eight transmissions of a confirmed frame with the data rate lowered every two [46]; LoRaWAN 1.0.4 sends a frame NbTrans times (1–15) at a fixed rate [47]; both are evaluated. The 1% duty cycle spaces frames [48]. A bulk frame carries 42 readings on NB-IoT, 18 at SF7, and 3 at SF12. Over a record of D days, the transmission energy per day and the lifetime on a battery of energy B are

$$E_{\mathrm{tx}} = \frac{1}{D}\sum_{a} e_a, \qquad Y = \frac{B}{365\,(E_{\mathrm{floor}} + E_{\mathrm{tx}})}, \tag{4}$$

where e_a is the energy of attempt a and E_{floor} is the daily sleep and sensing energy. Absolute energy depends on the platform and stack: an independent measurement on another board [37] reports 3× this model's per-message energy on LoRaWAN and 40× on NB-IoT (TCP without release assistance). The results are therefore ratios between strategies on the same radio; lifetimes are illustrative.

Five strategies are compared:

- Unconfirmed: one attempt per reading.
- Unconfirmed + back-off (NB-IoT): the fair baseline, because a node that searches for a cell in every slot of a gap wastes energy.
- ARQ: up to eight transmissions on LoRaWAN and three on NB-IoT.
- ARQ + buffer: a failed reading is buffered, and the node sends one small confirmed probe at a time, waiting

  $$w_n = \min\left(2^n,\, K\right) \text{ slots}, \qquad K = 6 \text{ h}, \tag{5}$$

  after the n-th failed probe and uploading the buffer in confirmed bulk frames once a probe succeeds.
- Batching: the buffer is uploaded every 1 or 6 h with the same back-off.

Replay: A slot in which a record delivered nothing is a slot in which the first attempt fails; what happens to later attempts in that slot is not in the record. Three models bracket it:

- Full slot: the link stays down for the whole slot (pessimistic for ARQ).
- Restoration: the link returns at a uniformly random time before the next delivered slot.

- Burst: in addition, gaps of up to an hour are radio-loss bursts through which retries usually pass (optimistic for ARQ).

Independent radio losses of 0.096 per attempt (LoED) can be added. The buffer is unlimited and free, unless stated otherwise. Supplementary Table S1 lists every input as measured, inferred, or assumed.

D. The server's real-time picture and the tolerances

Let $x(t)$ be the true reading of one parameter at slot t, $A(\tau)$ the set of slots whose readings have arrived by time τ, and $s^*(\tau) = \max A(\tau)$. The server shows

$$\hat{x}(\tau) = \begin{cases} x(\tau), & \tau \in A(\tau), \\ x(s^*) + \Delta(\tau), & \text{otherwise}, \end{cases} \tag{6}$$

where $\Delta(\tau) = x(\tau - 24\,\mathrm{h}) - x(s^* - 24\,\mathrm{h})$ if both readings have arrived and $\Delta(\tau) = 0$ otherwise. That is, the server shows the latest reading plus the site's own change over the same hours a day earlier (the own-diel estimator). The picture is scored as the share of slots with $|\hat{x}(\tau) - x(\tau)| \le \delta$. At the reference level L2, δ is 0.2 °C (T), 2% saturation (DO), 0.1 (pH), and 2% of the median (EC); L1 halves and L3 widens these. L2 lies at or inside the U.S. Geological Survey's accuracy rating "excellent" [49] for every parameter (Supplementary Section S6).

River data: River signals are approved USGS instantaneous values of T, EC, DO, and pH [50], with DO converted to percent saturation [51]. Section VI uses nine river chains (27 sites) in 2023. Section VII uses every candidate site with all four parameters in complete cycles for at least half of the year: 30 of 39 in 2023 (development) and 33 in 2024 (held out).

IV. How the Records Lose Data

In the water-quality deployments, most missing data were in long gaps. InWaterSense node 1 had 1,619 delivery gaps (timelines in Supplementary Fig. S1). Of these, 47.3% lasted a single slot but held only 4.7% of the missing readings, while 14 gaps of a day or more held 50.7% (95% CI 29.4%–64.6%), and gaps of 6 h or more held 68.5% (Fig. 1); the longest lasted 13.3 days. Weija, on another continent, showed the same structure: 82.9% of the missing conductivity readings fell

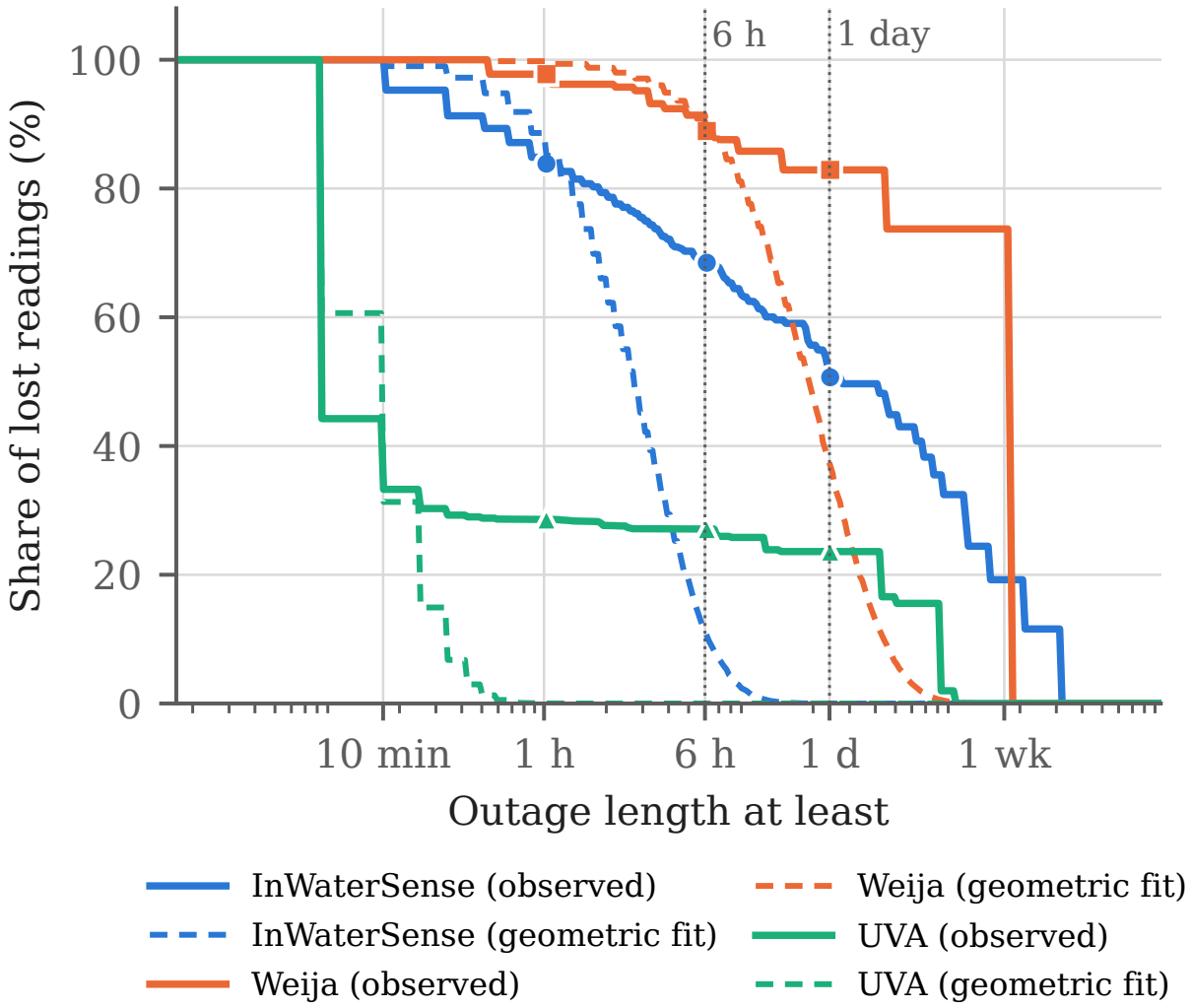


Fig. 1. Share of missing readings in gaps at least as long as the horizontal axis, $S(L)$ in (1) (solid), and as predicted by the geometric model (2) fitted to each record (dashed). InWaterSense node 1 (611-s slots), Weija conductivity (32.5-min slots), and UVA (5-min slots; radio and system losses together).

in gaps of a day or more (71.0%–100.0% across all streams), with the longest lasting 36.7 days. With only 3 day-long gaps, the Weija interval is wide (0.0%–93.5%); therefore, Weija serves as qualitative support.

The geometric model failed, and no tested model fitted. The geometric model (2) ranked last by AIC on every record (Table III), by 3,677 points on InWaterSense, 120 on Weija, and 43,915 on UVA. It also failed in the quantity that matters: fitted to InWaterSense, it put 11.4% of the missing readings in gaps of 6 h or more, against 68.5% observed, and less than 0.001%

Table III
AIC of gap-length models (lower is better; best per record in bold). UVA: all missing-frame runs, 5-min slots.

Model	InWaterSense	Weija	UVA
Geometric (memoryless)	10,537	341	76,881
Two-geometric mixture	7,107	236	**32,966**
Discrete lognormal	**6,859**	222	33,327
Discrete power law	6,863	**221**	33,180

in gaps of a day or more. It was also rejected absolutely on every record ($p \leq 0.005$). On the large records, every model was rejected, but the geometric model's largest distance between the fitted and observed distribution functions was 0.48 on InWaterSense, against 0.027 for the best alternative. No single alternative won everywhere; the mixture (3), with many short losses and a few long gaps, is the common thread. On InWaterSense, long gaps also clustered (Spearman $\rho = +0.27$ between the lengths of consecutive gaps); therefore, all simulations replay the real records rather than a fitted model.

The LoRaWAN networks showed short radio losses and rare system outages. On LoED, well-heard devices lost 9.6% of their frames to the radio, in short bursts: of 56,276 bursts, 84% were a single frame and 99% were at most five frames, and a loss after a loss had a probability of 0.25. UVA adds 16 months of continuous operation. The system was available 98.72% of the time; of the 58,211 missing frames, 71.6% were radio losses and 28.4% were system losses, and 83% of the system downtime fell in outages of a day or more. One gateway was down for 29.9 days without causing a single gateway loss, because every sensor was also heard by a second gateway, and rain had no detectable effect on link loss (9.18% dry, 9.21% rainy; Wilcoxon [52] $p = 0.64$).

In response to RQ1, in the four records, losses fell into short, nearly independent radio losses and rarer long delivery gaps or system outages. In the water-quality deployments, gaps of 6 h or more held most of the missing data (68.5% on InWaterSense, 88.9% on Weija). A geometric gap-length model missed this second regime almost entirely.

Table IV
Loss-handling strategies on the InWaterSense record (full-slot model, no added radio loss): transmission energy per day, E_{tx} in (4), per radio (NB-IoT: $T_{\mathrm{search}} = 10$ s), and the share of readings delivered and delivered within 1 h (LoRaWAN SF7; NB-IoT for the NB-IoT-only strategy). ARQ: LoRaWAN 1.0.2 behaviour. Back-off (5) capped at 6 h. –: not applicable.

	Energy (J/day)			Readings	
Strategy	NB-IoT	SF7	SF12	Delivered	Within 1 h
Unconfirmed, every reading	229.9	61.6	147.2	53.9%	53.9%
Unconfirmed + back-off (NB-IoT only)	59.5	–	–	47.1%	47.1%
ARQ (8 attempts LoRaWAN; 3 NB-IoT)	231.5	447.1	796.1	53.9%	53.9%
ARQ, 3 attempts (LoRaWAN)	–	175.5	354.3	53.9%	53.9%
Confirmed + buffer + back-off	63.8	52.7	158.7	100.0%	52.8%
Hourly batch + buffer + back-off	23.6	18.9	89.5	100.0%	52.0%
6-hourly batch + buffer + back-off	13.3	11.3	79.0	100.0%	11.2%

V. Retransmit or Backfill?

The InWaterSense and Weija records were replayed as the link state for NB-IoT (search times 2.2, 10, 60 s), LoRaWAN SF7, and LoRaWAN SF12, with and without independent radio losses: 20 configurations in all. Table IV shows InWaterSense under the full-slot model; ranges are over all configurations, and further results are given in Supplementary Section S4.

A. ARQ pays for losses it can hardly recover

Under the full-slot model, ARQ delivered exactly what unconfirmed reporting delivered, at 2.59×–7.35× its energy on LoRaWAN with eight transmissions. This is the pessimistic end of a bracket. If the link may return during a gap's last slot, retries recovered 0.1–4.6 percentage points of InWaterSense's readings on LoRaWAN; if, in addition, gaps of up to an hour are radio-loss bursts, they recovered 6.5–7.7 points. Against the 46 points that unconfirmed reporting lost, ARQ thus recovered under a fifth in every model, at 2.3×–7.3× the energy of unconfirmed reporting; on Weija, it recovered at most 0.9 points. NB-IoT ARQ recovered nothing because the modem reports that there is no network. LoRaWAN 1.0.4 (eight transmissions at a fixed rate) changed the price only slightly: 6.8× instead of 7.3× at SF7. Retransmission suits the radio regime, not the long gaps that hold most of the missing data.

B. Buffering buys completeness at a modest energy cost

On NB-IoT, searching for a cell in every slot of a gap cost 1.20×–6.53× the energy of backing off; thus, the fair NB-IoT baseline backs off. Against the fair baseline, confirmed reporting with a buffer and back-off delivered every generated reading at an energy change of -5.5% to $+15.3\%$ on NB-IoT and -14.6% to $+25.6\%$ on LoRaWAN. It was cheaper at SF7, where a confirmed success costs less than an unconfirmed report and the node stops transmitting into gaps, and more expensive at SF12. This completeness rests on three assumptions, each of which was tested:

1) The reading must exist: With up to half of the long-gap slots reassigned to a node that was off, buffering still delivered every generated reading, but only 83% of the expected readings on InWaterSense.
2) The buffer must hold the backlog: The longest backlog was 22 KB; 2 KB of RAM lost 24% of the readings and 64 KB none, and writing every reading to flash [53] cost at most 0.03 J/day.
3) The drain must leave room for live traffic: Limited to what the duty cycle leaves after one live frame per slot, the drain still delivered everything, but at SF12, a backlog persisted for 63 days in InWaterSense's worst period; therefore, live readings should go ahead of the backlog [6].

C. Batching is cheaper and costs little timeliness

Hourly batching with buffer and back-off delivered every reading, while the fair per-reading baseline used 1.33×–3.33× as much energy. Its share of readings within 1 h differed from that of unconfirmed reporting by only -3.3 to $+7.3$ points: under long gaps, the gaps, not the batch interval, decide how many readings arrive within the hour. The back-off cap trades energy against how long the server waits once the network is back. Probing every slot cost 2.2× the energy of $K = 6$ h at SF7, while raising K to 24 h saved only 12% more and tripled the 90th-percentile wait (20 to 61 h); $K = 6$ h sits at this knee. The price of batching is the timeliness of alarms and of the real-time picture of fast parameters, which Section VII addresses.

D. Many nodes and model uncertainty

A system outage ends for every node behind the gateway simultaneously. A discrete-event model of one EU868 gateway and up to 500 nodes (collisions with capture, eight demodulators, a half-duplex gateway, and gateway duty cycles; Supplementary Table S2) showed two effects. Without randomisation, the deterministic back-off kept the nodes synchronised: from 50 nodes at SF7, almost none recovered within 72 h. With probes delayed randomly by up to an hour, every SF7 node recovered, the last within 42 h, at up to 12.5× the energy of a node alone. At SF12, the gateway's acknowledgements ran out: its first-window sub-band reached its 1% limit, up to 29% of the received frames went unacknowledged, and after a one-day outage, only 93% of 100 nodes recovered within 72 h. The per-node results hold for cells with tens of nodes.

With every measured energy component multiplied by an independent factor between 1/2 and 2 (100 draws), ARQ cost more than unconfirmed reporting on LoRaWAN in 100% of the draws. Hourly batching was cheaper than the fair baseline in at least 98% of them at SF7 and on NB-IoT, but only in 85%–93% at SF12. The buffering energy relative to the fair baseline ranged from 0.50× to 2.08×, and its sign is set by the ratio of confirmed to unconfirmed energy, which can only be settled by measuring the target hardware.

In response to RQ2, ARQ recovers only the short radio losses under any timing model, and on LoRaWAN, it multiplies energy. Buffering with back-off recovers every generated reading, given tens of kilobytes of storage, at an energy close to that of recovering nothing in the model, and hourly batching is the energy-efficient way to do it at SF7 and on NB-IoT. Many nodes recovering simultaneously need randomised probes, and large SF12 cells are limited by the gateway's acknowledgements.

VI. Can Reconstruction Replace Delivery?

If a server could estimate missing readings sufficiently well, it would not need them delivered. This was tested on nine river chains, i.e., sets of USGS sites along the same river (27 sites), with each pair of sites serving as target and neighbour in both orders. On a 1-h grid, gaps of 1, 3, 6, 12, 24, 48, 72 h were simulated at the target in the last 80% of 2023. Two families of estimators were compared:

Table V
Reconstruction on nine river chains: median over ordered site pairs of the median maximum absolute error over a gap (T in °C, DO in %sat, pH in pH units, EC in μS/cm), and the share of pairs in which the neighbour improves on the target's own data. Real time: best neighbour method vs the better of persistence and own diel. Bridging: interpolation + neighbour vs interpolation.

		Real time				Bridging		
Param.	Gap (h)	Persis-tence	Own diel	Best neighbour	Neighbour wins	Interpolation	Interp. + neighbour	Neighbour wins
T	1	0.07	0.05	0.05	23%	0.03	0.03	19%
T	24	0.97	0.70	0.65	72%	0.80	0.56	93%
DO	1	0.67	0.49	0.58	19%	0.26	0.34	13%
DO	24	9.07	4.33	5.13	30%	7.75	6.27	70%
pH	1	0.000	0.025	0.018	7%	0.013	0.013	9%
pH	24	0.175	0.125	0.149	9%	0.172	0.172	38%
EC	1	0.50	0.50	0.66	16%	0.25	0.27	16%
EC	24	7.50	8.25	7.50	23%	4.44	4.42	25%

- Real-time estimators use data up to the gap and the neighbour during it: persistence; the own-diel estimator of (6); the neighbour's scaled change (lag and slope fitted on the first 20% of the year); and own diel plus the part of the neighbour's change that its own diel pattern does not explain.
- Bridging estimators also know the value after the gap, as after a backfill: linear interpolation, and interpolation plus the neighbour's departure from its own line.

The equations are given in Supplementary Section S5. The error on a gap is its maximum absolute error; confidence intervals resample whole chains, and pairs within them.

In real time, a site's own diel pattern was the estimator to beat (Table V). For day-long gaps, it beat persistence in 77% of pairs for DO, and a neighbour improved on the better own-data estimator in only 30% of pairs for DO, 9% for pH, and 23% for EC; temperature, with its shared weather driver, was the exception (72%). When bridging, the neighbour helped T (93% of pairs) and DO (70%), but not pH or EC.

A naive baseline overestimates the value of neighbours. Against persistence alone, a neighbour won in 77% of DO pairs, cutting the median error from 9.07 to 5.13 %sat; however, the own-diel estimator already reached 4.33 %sat. Much of what a naive test credits to spatial correlation is the shared day–night cycle. The fitted relation also did not follow the flow: a downstream

neighbour got a positive lag approximately as often as an upstream one (64% vs 68% for DO); therefore, the lag tracks shared timing rather than travel time. Evaluations of spatio-temporal reconstruction on rivers therefore need a per-site diel baseline and spatial structure from the river network [27], [30].

No estimator kept day-long gaps reliably within tolerance. For 1-h gaps, the better own-data estimator kept 97% of gaps within the L2 tolerance; for 24-h gaps, the best of all six kept 31% (95% CI 18%–49%), from 9% for T to 49% for pH.

A. Contemporary deep imputers

To test whether this is a limit of reconstruction or of simple estimators, SAITS, a self-attention imputer [28], and StemGNN, a graph neural network [26], were trained per chain on the first half of 2023, with whole-site gaps of 1–72 h among their training masks. They were tested on whole-site gaps in the second half, on the same 48,455 cases as the simple estimators (protocol in Supplementary Section S5). They kept fewer gaps within tolerance: 65% (SAITS) and 62% (StemGNN) of 1-h gaps when bridging, against 97% for interpolation, and 25% and 23% of day-long gaps, against 31%. In real time, they kept 18% and 16% of day-long gaps, against 29% for own diel. They helped in one place: when bridging a day-long gap, the median error of SAITS was 4.3 %sat for DO and 0.126 for pH, against 6.7 and 0.148 for interpolation. No estimator, simple or deep, kept more than 55% of the day-long gaps of any parameter within tolerance.

In response to RQ3, the evaluated estimators, simple or deep, cannot replace delivery for the day-long gaps that hold most of the missing data. Reconstruction plays the role of the interim estimate between deliveries, which is the role it plays in the design of Section VII.

VII. A Reporting Design and its Evaluation

Sections IV–VI point to a design: buffer and back off through gaps, batch routine data, and show the site's own diel estimate between deliveries. Batching leaves two weaknesses: alarms and the real-time picture of fast parameters. The design closes them with two triggers that send a batch early.

A. The design

Event-driven batching buffers every reading and uploads the buffer once per hour in confirmed bulk frames. When the node's event detector raises an alarm, the buffer is uploaded immediately. A failed upload puts the node into an outage mode, in which an alarm triggers an immediate probe outside the back-off schedule (5) (Supplementary Fig. S2 illustrates the behaviour).

Bounded event-driven batching adds a bounded-error trigger (Algorithm 1). Because every upload is confirmed, the node knows which readings have reached the server and when; therefore, it can compute the value $\hat{x}_j(t)$ that the server shows for each parameter j by evaluating (6) on those readings. It uploads immediately if

$$\max_j \ \frac{|x_j(t) - \hat{x}_j(t)|}{\delta_j} > 1, \tag{7}$$

that is, if any parameter's true reading differs from what the server shows by more than its tolerance δ_j. This is the dual-prediction mechanism [38], [39], used to bound the error between batches rather than to suppress reports. Three details matter:

1) The trigger is not consulted in outage mode, where the estimate may drift far and probing would waste energy on a dead link; only alarms probe.
2) The node's copy of the server must model when readings arrive. The duty cycle spaces the frames of a long upload; therefore, a reading counts only after its frame's arrival, and the newest valid reading goes in the first frame.
3) The node's knowledge is only as good as its acknowledgements. On NB-IoT, the confirmation is the application's reply; on LoRaWAN, it comes from the network server, and forwarding to the application server, which UVA shows can fail, is not covered. A variant closes this gap by keeping the bound for both possible server states until an application acknowledgement returns in a later downlink (Section VII-C).

B. Evaluation protocol

Two phases: The design was developed on 2023 data and tested on held-out 2024 data. In development, hypotheses H1–H4 and their decision rules were written before their runs; after

Algorithm 1 Bounded event-driven batching: node, one slot

append the slot's reading x_t to the buffer
if in outage mode **then**
    if alarm at t **or** back-off timer expired **then**
        probe; on success leave outage mode and upload the buffer
    end if
    return
end if
$\hat{x}(t) \leftarrow$ server's estimate (6), from readings whose arrival time is $\leq t$
if alarm at t **or** trigger (7) **or** hour boundary **then**
    upload the buffer (newest valid reading first), confirmed bulk frames
    on failure enter outage mode, back-off (5)
end if

H4 failed, H5–H10 were written for the bounded-error trigger before it was implemented or run. Five deviations were recorded when made (D1–D5, Supplementary Section S6). The last, found during this revision, was a floating-point tie at the tolerance, and all development results are from a rerun with it fixed. For the held-out phase, the final design, the analysis code, its decision rules, and the site-eligibility rule were frozen, and the code was identified by cryptographic hashes before any 2024 data were downloaded. The plans were kept with the code; they were not registered with a third party.

Setting: The evaluation used 30 sites in 2023 and 33 in 2024 (29 in both), sampled every 5–60 min.

- Links: no outage, and the InWaterSense, Weija, and UVA records, mapped onto each site's time grid with a random offset per site (UVA as system outages plus independent radio losses).
- Radios: NB-IoT, LoRaWAN SF7, and LoRaWAN SF12.
- Strategies: unconfirmed, ARQ + buffer, hourly batching, event-driven batching, and bounded event-driven batching (L2, unless stated otherwise).
- Events: steps and ramps of 5 and 8 robust standard deviations, lasting 3 h, injected one at a time into the real series (7,548 in 2024). Of these, 99.7% stay within physically plausible ranges, and real 3-h excursions of their size occur at most sites for T, DO, and EC, and less often for pH. All strategies use the same causal CUSUM detector, fixed before any evaluation, which detected 58.2% of the events with a median of 0.8 false alarms per

Table VI
Hypotheses, decision rules, and verdicts. Each rule applies to the median site of every link–radio (12) or link–radio–parameter (48) combination and passes only if it holds in all. Written: P, before the development runs; A, after H4 failed, before the runs concerned. 2023: development rerun with all deviations fixed; 2024: held out, judged by the frozen code. Event: event-driven batching; Bounded: bounded event-driven batching.

		Claim	Rule	2023 (development)	2024 (held out)
H1	P	Link up: Event raises alarms as fast as per-reading reporting	Event − ARQ+buffer alarm latency $\leq$ 1 slot	Pass, 12/12	Pass, 12/12
H2	P	Event costs approximately as much as batching, much less than per-reading ARQ	Event / batch $\leq$ 1.10 and ARQ+buffer / Event $\geq$ 1.3	Pass, 12/12	Pass, 12/12
H3	P	Onset in a gap: buffering delivers late, unconfirmed loses	Descriptive	As expected	As expected
H4	P	Event's picture is as good as per-reading reporting's	Event $\geq$ ARQ+buffer − 5 pp	Fail, 38/48	Fail, 39/48
H5	A	Bounded's picture is as good as per-reading reporting's	Bounded $\geq$ ARQ+buffer − 1 pp	Pass, 48/48	Pass, 48/48
H6	A	Bounded costs no more than per-reading ARQ	Bounded $\leq$ ARQ+buffer energy	Pass, 12/12	Pass, 12/12
H7	A	The trigger does not delay alarms	Bounded − Event alarm latency $\leq$ 1 slot	Pass, 12/12	Pass, 12/12
H8	A	Newest-first packing makes the picture exact with the link up	100% within tolerance, no outage, every site	Fail as written; pass after D4	Pass
H9	A	H5 holds at tolerances L1 and L3	As H5	Pass, 48/48, 48/48	Pass, 48/48, 48/48
H10	A	H6 holds at tolerances L1 and L3	As H6	Pass, 12/12, 12/12	Pass, 12/12, 12/12

parameter per 30 days (Supplementary Section S6).

The strategies were compared on three metrics:

- Alarm latency, from event onset to arrival at the server (detected events, link up at onset).
- Transmission energy (4), including alarm and false-alarm uploads.
- The picture, i.e., the share of slots within tolerance on the series without events.

Each is the median over sites per link and radio (12 combinations; 48 with the four parameters), with 95% confidence intervals from resampling sites. Table VI lists the hypotheses and verdicts.

C. Results

Alarms (H1, H7): With the link up, event-driven batching raised alarms as fast as per-reading ARQ in both years (Table VII); the median latency of approximately an hour is the detector's own. Plain hourly batching was slower in every combination, by 0.43–0.50 h in 2024, and the

Table VII
Held-out year (2024): alarm latency with the link up (median over sites, h) and transmission energy (median over sites, J/day); LoRaWAN SF7. ARQ: ARQ + buffer.

	Alarm latency (h)				Energy (J/day)			
Link	ARQ	Batch	Event	Bounded	ARQ	Batch	Event	Bounded
No outage	1.00	1.25	1.00	1.00	34.2	9.0	9.0	11.4
UVA	1.00	1.25	1.00	1.00	36.1	9.6	9.7	12.2
Weija	1.00	1.25	1.00	1.00	32.2	10.7	10.9	13.0
InWaterSense	1.00	1.50	1.00	1.00	37.1	18.4	18.7	21.4

bounded-error trigger did not delay the alarms. Counting all injected events, the server knew 34% within an hour with event-driven batching on the no-outage link, as with per-reading reporting, against 21% with hourly batching; the ceiling is detection, not reporting.

Energy (H2, H6): Alarms were rare (a median of 0.13 per node per day); therefore, event-driven batching cost 1.00×–1.02× as much as plain batching. The bounded-error trigger added uploads between batches, at 1.05×–1.28× the energy of plain batching, and per-reading ARQ still used 1.34×–2.55× as much energy as the bounded design (every site-resampled 95% interval above 1.28×; 2023: 1.32×–2.51×).

Picture (H4, H5): Plain event-driven batching failed H4 in both years, for DO only: an estimate up to an hour old leaves the tolerance between batches, and the share of slots within tolerance differed from that of per-reading reporting by -9.0 to -6.9 points. The failures occurred with no outage at all; they are a property of hourly batching. (In the first development run, H4 also failed for pH; those failures were caused by the tolerance tie, not by batching.) The bounded-error trigger removed the failure: its share differed from that of per-reading reporting by -0.01 to $+0.56$ points for every parameter, link, and radio (Fig. 2), and with the link up, it was within tolerance at every slot (H8, after the packing-order fix D4). The bound costs more at tighter tolerances, yet per-reading ARQ used 1.20×–1.57× the bounded design's energy at L1 and 1.40×–3.47× at L3 (H9, H10).

Events in a gap (H3): Each strategy must wait for a link. Unconfirmed reporting learned at most 7.0% of the detected events whose onset fell in a gap; the buffering strategies learned at least 98.8% after the gap, and the alarm probe made the event-driven designs faster than ARQ + buffer on the water-quality records.

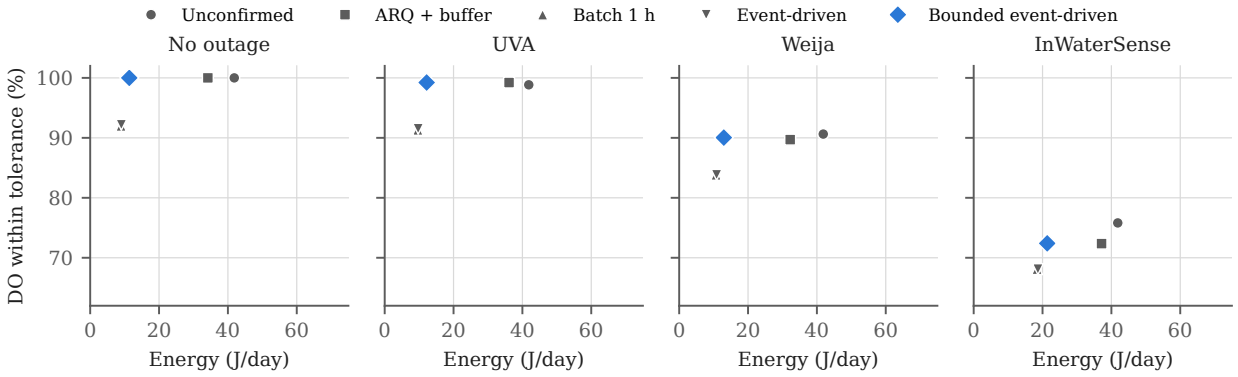


Fig. 2. Held-out year (2024): energy against the real-time picture of DO (share of slots within the L2 tolerance), LoRaWAN SF7, median over sites, one panel per link, and all panels on the same vertical scale. Batch 1 h and event-driven batching nearly coincide, and the bounded design reaches the picture of per-reading reporting at a fraction of its energy.

Acknowledgement boundary: If a LoRaWAN node may not trust the network server's acknowledgement and keeps the bound for both possible server states, the design cost 3% to 18% more energy on the development sites. Its picture stayed within -0.02 points of per-reading reporting, and per-reading ARQ still used $1.29\times$–$2.00\times$ as much energy.

The answer to RQ4 is yes, with the link up and on data not used for the design of the scheme. Bounded event-driven batching delivers every generated reading, raises alarms as fast as per-reading reporting, and keeps the picture within tolerance as often, while per-reading ARQ used $1.34\times$–$2.55\times$ as much energy. No scheme fixes the gap itself: data and events during a gap reach the server only when the link returns.

VIII. Discussion

A. *Practical guidance*

The results lead to six recommendations:

1) Determine which regime loses the data: Short radio losses and long delivery gaps call for different remedies. Frame counters, a log of gateway and server downtime, and the node's own status are sufficient to distinguish between them; models that assume geometric gap lengths should be checked against such a log.

2) Do not fight long gaps with retransmissions: Confirm, buffer, and back off; on NB-IoT, back off even without a buffer, because searching for a cell in every slot of a gap cost up to 6.53× the energy of backing off.
3) Give the buffer sufficient room and keep the backlog from blocking live traffic: Tens of kilobytes held the longest backlog in these records. At SF12, the duty cycle limits recovery; therefore, live readings should go ahead of the backlog. Where many nodes share a gateway, recovery probes should be randomised and backlogs acknowledged in chunks.
4) Batch, and let the batch leave early when it matters: Hourly batching delivers every reading for a fraction of per-reading energy at SF7 and on NB-IoT. An alarm trigger restores timeliness, and a bounded-error trigger that mirrors the server's estimate restores the real-time picture, with the tolerance as the operator's energy control. The node must model when its data arrive and, on LoRaWAN, whether the application has them.
5) Judge reconstruction against the site's own diel pattern, the best simple real-time estimator for DO and pH: A test against the last value alone overstates what neighbours and models add.
6) Engineer the gaps themselves: No reporting scheme shortens a gap. Gateway diversity masked a month-long gateway outage at UVA, and watchdog resets bound silent gateway failures [6], [7].

B. Validity and limitations

The main limitations are as follows:

- Records: The loss evidence comes from four records and supports statements about them, not a law for all deployments; Weija serves as qualitative support. The cause of a gap could be checked only partly; therefore, buffering's recovery is an upper bound.
- Energy without hardware: No hardware was built for this study. The model rests on published measurements of specific modules, and an independent measurement on another board differs severalfold in absolute terms. The claims are therefore ratios on the same radio, tested against a factor-of-two uncertainty in every component (Section V-D). At SF12, the

advantage of batching, and everywhere, the sign of buffering's small energy premium, need hardware measurement, which is the most important next step.

- Model scope: Clock faults and storage failure are not simulated, and the multi-node model of one gateway shows mechanisms (synchronised probes, acknowledgement capacity) rather than numbers for a particular network.
- Events and rivers: Events are synthetic steps and ramps of realistic size, and the detector catches slow ramps poorly (13% of 5σ ramps); the latency comparisons concern detected events. The river data cover two years in one country, and the held-out year shares most sites with the development year (29 of 33); therefore, it tests the design on new data rather than on new rivers.
- Pre-specification: Hypotheses were written before their runs, and the confirmatory code was frozen before the held-out data were downloaded, but neither was registered with a third party. Every verdict is reported, including the failures and the defect found during this revision.

C. Conclusion

In the four records studied, data were lost in two regimes: short radio losses and rarer long delivery gaps that, in the water-quality deployments, held most of the missing data. Retransmission spends energy on gaps it cannot bridge, and the evaluated reconstruction estimators, simple or deep, kept most day-long gaps outside the tolerance. Buffering with back-off delivers every generated reading at an energy close to that of not recovering at all, and hourly batching makes it cheaper still. A design that batches hourly, sends alarms immediately, and leaves early whenever the server's estimate would leave tolerance matched per-reading reporting on alarm latency and on the real-time picture on a held-out year, while per-reading ARQ used $1.34\times$–$2.55\times$ as much energy. What remains is the delivery gap itself, which can only be shortened by network engineering. Future work includes hardware measurements, longer delivery records from tropical deployments, the energy of NB-IoT cell search without a network, and recovery across several gateways.

Data and Code Availability

The USGS [50], InWaterSense [41], LoED [43], and UVA [42] data are public. The Weija records were provided by the deployment's operators [2]; their raw values are not redistributed, but the delivery trace derived from them (the time of every record of every stream, and whether it counts as a genuine delivery) is. The code, configurations, seeds, environment lock files, data-acquisition scripts, input checksums, the frozen confirmatory plan with the hashes of the frozen code, and the derived traces are archived at Zenodo [54] (doi:10.5281/zenodo.23455071). Every number in this paper and its supplement is generated from the result files by a script that records its source.

Acknowledgment

This study received no external funding.

References

[1] K. S. Adu-Manu, C. Tapparello, W. Heinzelman, F. A. Katsriku, and J.-D. Abdulai, "Water quality monitoring using wireless sensor networks: Current trends and future research directions," *ACM Transactions on Sensor Networks*, vol. 13, no. 1, pp. 1–41, 2017.

[2] K. S. Adu-Manu, F. A. Katsriku, J.-D. Abdulai, and F. Engmann, "Smart river monitoring using wireless sensor networks," *Wireless Communications and Mobile Computing*, vol. 2020, pp. 1–19, 2020, art. no. 8897126.

[3] F. Ahmedi and L. Ahmedi, "Dataset on water quality monitoring from a wireless sensor network in a river in Kosovo," *Data in Brief*, vol. 44, p. 108486, 2022.

[4] T. Le Dinh, W. Hu, P. Sikka, P. Corke, L. Overs, and S. Brosnan, "Design and deployment of a remote robust sensor network: Experiences from an outdoor water quality monitoring network," in *Proc. 32nd IEEE Conf. Local Computer Networks (LCN)*. IEEE, 2007, pp. 799–806.

[5] K. Langendoen, A. Baggio, and O. Visser, "Murphy loves potatoes: Experiences from a pilot sensor network deployment in precision agriculture," in *Proc. 20th IEEE Int. Parallel and Distributed Processing Symp. (IPDPS)*. IEEE, 2006, p. 8 pp.

[6] F. Ingelrest, G. Barrenetxea, G. Schaefer, M. Vetterli, O. Couach, and M. Parlange, "SensorScope: Application-specific sensor network for environmental monitoring," *ACM Transactions on Sensor Networks*, vol. 6, no. 2, pp. 17:1–17:32, 2010.

[7] M. Brzozowski, M. Frohberg, and P. Langendoerfer, "Real-world deployments of sensor networks: Practical lessons for researchers," in *Wired/Wireless Internet Communications (WWIC 2018), LNCS vol. 11618*. Springer International Publishing, 2018, pp. 42–53.

[8] P. P. Puluckul and M. Weyn, "Experiences with sub-arctic sensor network deployment," in *Proc. IEEE Wireless Communications and Networking Conf. (WCNC)*. IEEE, 2025, pp. 1–6.

[9] M. Zennaro, E. Pietrosemoli, A. Soltobaev, T. Sultanov, and B. Moldobekov, "From mountains to data: low-cost weather stations in Kyrgyzstan's challenging terrain," *Frontiers in Communications and Networks*, vol. 6, 2025, art. no. 1505375.

[10] B. Sinopoli, L. Schenato, M. Franceschetti, K. Poolla, M. Jordan, and S. Sastry, "Kalman filtering with intermittent observations," *IEEE Transactions on Automatic Control*, vol. 49, no. 9, pp. 1453–1464, 2004.

[11] K. Huang, W. Liu, Y. Li, and B. Vucetic, "To retransmit or not: Real-time remote estimation in wireless networked control," in *Proc. IEEE Int. Conf. Communications (ICC)*. IEEE, 2019, pp. 1–7.

[12] K. Huang, W. Liu, M. Shirvanimoghaddam, Y. Li, and B. Vucetic, "Real-time remote estimation with hybrid ARQ in wireless networked control," *IEEE Transactions on Wireless Communications*, vol. 19, no. 5, pp. 3490–3504, 2020.

[13] L. Kong, M. Xia, X.-Y. Liu, G. Chen, Y. Gu, M.-Y. Wu, and X. Liu, "Data loss and reconstruction in wireless sensor networks," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 11, pp. 2818–2828, 2014.

[14] A. Cini, I. Marisca, and C. Alippi, "Filling the G_ap_s: Multivariate time series imputation by graph neural networks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022, arXiv:2108.00298.

[15] I. Marisca, A. Cini, and C. Alippi, "Learning to reconstruct missing data from spatiotemporal graphs with sparse observations," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[16] Q. Chang, D. Tao, J. Wang, and R. Gao, "Deep compressed sensing based data imputation for urban environmental monitoring," *ACM Transactions on Sensor Networks*, vol. 20, no. 1, pp. 1–21, 2023, art. no. 17.

[17] J. Zhao and R. Govindan, "Understanding packet delivery performance in dense wireless sensor networks," in *Proc. 1st Int. Conf. Embedded Networked Sensor Systems (SenSys)*. ACM, 2003, pp. 1–13.

[18] L. Bhatia, M. Breza, R. Marfievici, and J. A. McCann, "LoED: The LoRaWAN at the edge dataset," in *Proc. 3rd Workshop on Data: Acquisition To Analysis (DATA '20)*. ACM, 2020, pp. 7–8.

[19] F. Nikseresht, V. A. L. Sobral, J. L. Goodall, and B. Campbell, "Dataset: Long-term LoRaWAN communication metadata from an urban deployment," in *Proc. 12th ACM Int. Conf. Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys '25)*. ACM, 2025, pp. 412–417.

[20] R. D. Yates, Y. Sun, D. R. Brown, S. K. Kaul, E. Modiano, and S. Ulukus, "Age of information: An introduction and survey," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 5, pp. 1183–1210, 2021.

[21] B. Sun and N. Pappas, "Resource-constrained semantic-aware remote estimation with overlapping sensor coverage," in *Proc. ACM MobiHoc*, 2026, to appear; arXiv:2609.07563.

[22] P. J. Marcelis, V. Rao, and R. V. Prasad, "DaRe: Data recovery through application layer coding for LoRaWAN," in *Proc. 2nd Int. Conf. Internet-of-Things Design and Implementation (IoTDI)*. ACM, 2017, pp. 97–108.

[23] C. Luo, F. Wu, J. Sun, and C. W. Chen, "Compressive data gathering for large-scale wireless sensor networks," in *Proc. 15th Annu. Int. Conf. Mobile Computing and Networking (MobiCom)*. ACM, 2009, pp. 145–156.

[24] J. Cheng, Q. Ye, H. Jiang, D. Wang, and C. Wang, "STCDG: An efficient data gathering algorithm based on matrix completion for wireless sensor networks," *IEEE Transactions on Wireless Communications*, vol. 12, no. 2, pp. 850–861, 2013.

[25] X. Piao, Y. Hu, Y. Sun, B. Yin, and J. Gao, "Correlated spatio-temporal data collection in wireless sensor networks based on low rank matrix approximation and optimized node sampling," *Sensors*, vol. 14, no. 12, pp. 23 137–23 158, 2014.

[26] D. Cao, Y. Wang, J. Duan, C. Zhang, X. Zhu, C. Huang, Y. Tong, B. Xu, J. Bai, J. Tong, and Q. Zhang, "Spectral temporal

graph neural network for multivariate time-series forecasting," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 17 766–17 778.

[27] B. Jing, D. Zhou, K. Ren, and C. Yang, "Causality-aware spatiotemporal graph neural networks for spatiotemporal time series imputation," in *Proc. 33rd ACM Int. Conf. Information and Knowledge Management (CIKM)*. ACM, 2024, pp. 1027–1037.

[28] W. Du, D. Côté, and Y. Liu, "SAITS: Self-attention-based imputation for time series," *Expert Systems with Applications*, vol. 219, p. 119619, 2023.

[29] T. Nie, G. Qin, W. Ma, Y. Mei, and J. Sun, "ImputeFormer: Low rankness-induced transformers for generalizable spatiotemporal imputation," in *Proc. 30th ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD)*. ACM, 2024, pp. 2260–2271.

[30] E. E. Peterson and J. M. Ver Hoef, "A mixed-model moving-average approach to geostatistical modeling in stream networks," *Ecology*, vol. 91, no. 3, pp. 644–651, 2010.

[31] P. McDonald, D. Geraghty, I. Humphreys, S. Farrell, and V. Cahill, "Sensor network with delay tolerance (SeNDT)," in *2007 16th International Conference on Computer Communications and Networks*. IEEE, 2007, pp. 1333–1338.

[32] M. J. Murphy, Ø. Tveito, E. F. Kleiven, I. Rais, E. M. Soininen, J. M. Bjørndalen, and O. Anshus, "Experiences building and deploying wireless sensor nodes for the arctic tundra," in *Proc. IEEE/ACM 21st Int. Symp. Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 2021, pp. 376–385.

[33] L. Casals, B. Mir, R. Vidal, and C. Gomez, "Modeling the energy performance of LoRaWAN," *Sensors*, vol. 17, no. 10, p. 2364, 2017.

[34] F. Michelinakis, A. S. Al-Selwi, M. Capuzzo, A. Zanella, K. Mahmood, and A. Elmokashfi, "Dissecting energy consumption of NB-IoT devices empirically," *IEEE Internet of Things Journal*, vol. 8, no. 2, pp. 1224–1242, 2021.

[35] A. Sørensen, H. Wang, M. J. Remy, N. Kjettrup, R. B. Sørensen, J. J. Nielsen, P. Popovski, and G. C. Madueño, "Modeling and experimental validation for battery lifetime estimation in NB-IoT and LTE-M," *IEEE Internet of Things Journal*, vol. 9, no. 12, pp. 9804–9819, 2022.

[36] A. Azari, C. Stefanovic, P. Popovski, and C. Cavdar, "On the latency-energy performance of NB-IoT systems in providing wide-area IoT connectivity," *IEEE Transactions on Green Communications and Networking*, vol. 4, no. 1, pp. 57–68, 2020.

[37] S. Ugwuanyi, G. Paul, and J. Irvine, "Survey of IoT for developing countries: Performance analysis of LoRaWAN and cellular NB-IoT networks," *Electronics*, vol. 10, no. 18, p. 2224, 2021, art. no. 2224.

[38] G. M. Dias, B. Bellalta, and S. Oechsner, "A survey about prediction-based data reduction in wireless sensor networks," *ACM Computing Surveys*, vol. 49, no. 3, pp. 1–35, 2016.

[39] S. Santini and K. Römer, "An adaptive strategy for quality-based data reduction in wireless sensor networks," in *Proc. 3rd Int. Conf. Networked Sensing Systems (INSS)*, 2006.

[40] D. Arnaiz, F. Moll, E. Alarcón, and X. Vilajosana, "Energy and relevance-aware adaptive monitoring method for wireless sensor nodes with hard energy constraints," *Integration*, vol. 94, p. 102097, 2024.

[41] L. Ahmedi, "InWaterSense dataset: Data from a wireless sensor network on water quality monitoring in a river in Kosovo," Mendeley Data, V1, 2021.

[42] F. Nikseresht, V. A. L. Sobral, J. L. Goodall, and B. Campbell, "Long-term LoRaWAN communication metadata from an urban deployment," University of Virginia Dataverse (LibraData), 2025.

[43] L. Bhatia, M. Breza, R. Marfievici, and J. A. McCann, "LoED: The LoRaWAN at the edge dataset," Zenodo, 2020.

[44] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[45] *Waspmote Plug & Sense! Technical Guide (product line v15)*, Libelium Comunicaciones Distribuidas, 2020, document version v9.2.

[46] LoRa Alliance Technical Committee, "LoRaWAN specification v1.0.2," LoRa Alliance, Tech. Rep., 2016, sec. 18.4, Data-rate adaptation during message retransmissions.

[47] ——, "LoRaWAN L2 1.0.4 specification (TS001-1.0.4)," LoRa Alliance, Tech. Rep., 2020, sec. 4.3.1.3, Retransmission procedure; Sec. 5.3, LinkADRReq (NbTrans).

[48] LoRa Alliance Technical Committee Regional Parameters Workgroup, "RP002-1.0.3 LoRaWAN regional parameters," LoRa Alliance, Tech. Rep., 2021.

[49] R. J. Wagner, R. W. Boulger, C. J. Oblinger, and B. A. Smith, *Guidelines and standard procedures for continuous water-quality monitors: Station operation, record computation, and data reporting*, 2006.

[50] U.S. Geological Survey, "USGS water data for the nation (National Water Information System)," 2024, instantaneous values for 2023, accessed Oct. 2026.

[51] B. B. Benson and D. Krause, "The concentration and isotopic fractionation of oxygen dissolved in freshwater and seawater in equilibrium with the atmosphere," *Limnology and Oceanography*, vol. 29, no. 3, pp. 620–632, 1984.

[52] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, p. 80, 1945.

[53] *W25Q128JV 3V 128M-bit Serial Flash Memory, Datasheet, Rev. F*, Winbond Electronics, 2018, sec. 9.4 (DC characteristics) and AC electrical characteristics.

[54] S. Azongo, "river-telemetry-loss-handling, version v1.0.0: code, configurations and derived data," Zenodo, software, 2026.